Árboles de búsqueda binaria

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

February 16, 2026

Descripción de la unidad

Implementación y análisis de árboles de búsqueda binaria y árboles balanceados.

Universidad Pontificia Bolivariana

Contents

1 Árboles Binarios de Búsqueda **3**
1.1 Definición . . . 3
1.1.1 Aplicaciones Prácticas . . . 3
1.1.2 Construcción del BST . . . 3
1.1.3 Ejemplo Visual . . . 4
1.2 Operaciones en el BST . . . 4
1.2.1 Get (Búsqueda) . . . 4
1.2.2 Put (Inserción) . . . 5
1.2.3 Delete (Eliminación) . . . 5
1.2.4 Otras operaciones . . . 6
1.3 Recorrido de Árboles . . . 7
1.3.1 Preorden (Raíz-Izquierda-Derecha) . . . 7
1.3.2 Inorden (Izquierda-Raíz-Derecha) . . . 7
1.3.3 Postorden (Izquierda-Derecha-Raíz) . . . 8
1.4 Consideraciones de Implementación . . . 8
1.4.1 Mejores Prácticas . . . 8
1.4.2 Errores Comunes . . . 8
1.5 Optimizaciones . . . 9
1.5.1 Árboles Balanceados . . . 9
1.5.2 Casos de Uso y Selección . . . 9

2 Ejercicios y Práctica **9**
2.1 Ejercicios Teóricos . . . 9
2.2 Ejercicios Prácticos . . . 10
2.2.1 Construcción de BST . . . 10
2.2.2 Implementación . . . 10
2.3 Problemas de Desafío . . . 10
2.4 Soluciones y Pistas . . . 11
2.4.1 Pistas para los Ejercicios Teóricos . . . 11
2.4.2 Pistas para los Ejercicios Prácticos . . . 11
2.4.3 Pistas para los Problemas de Desafío . . . 11

3 Árboles de búsqueda 2-3 **12**
3.1 Definición . . . 12
3.2 Estructura Visual . . . 12
3.3 Búsqueda en árboles 2-3 . . . 12
3.4 Inserción en árboles 2-3 . . . 13
3.5 Eliminación en árboles 2-3 . . . 14
3.6 Ventajas y Desventajas . . . 14
3.7 Pseudocódigo de Operaciones Básicas . . . 14
3.8 Complejidad Algorítmica . . . 14
3.9 Relación con Árboles Rojo-Negros . . . 15

4 Árboles Rojo-Negros **16**
4.1 Definición . . . 16
4.2 Relación con los Árboles 2-3 . . . 16
4.3 Implementación del Árbol Rojo-Negro . . . 17
4.3.1 Representación del Color . . . 17
4.3.2 Rotaciones . . . 18
4.3.3 Inserción . . . 19
4.3.4 Eliminación . . . 19
4.4 Eficiencia de Operaciones en Árboles Rojo-Negros . . . 19
4.5 Consideraciones Prácticas . . . 21
4.6 Ejercicios Recomendados . . . 21

5 Otros tipos de árboles de búsqueda **22**
5.1 Árboles AVL . . . 22
5.1.1 Características Principales . . . 22
5.1.2 Ventajas y Desventajas . . . 22
5.1.3 Casos de Uso . . . 23
5.2 Árboles B (B-Trees) . . . 23
5.2.1 Características Principales . . . 23
5.2.2 Ventajas y Desventajas . . . 23
5.2.3 Casos de Uso . . . 24
5.3 Árboles B+ (B+ Trees) . . . 24
5.3.1 Características Principales . . . 24
5.3.2 Diferencias con Árboles B . . . 25
5.3.3 Ventajas y Desventajas . . . 25
5.3.4 Casos de Uso . . . 25
5.3.5 Referencias Adicionales . . . 26
5.4 Comparación de Estructuras de Árboles de Búsqueda . . . 26
5.4.1 Guía de Selección . . . 26
5.4.2 Resumen de Rendimiento . . . 27

1 Árboles Binarios de Búsqueda

1.1 Definición

Un **árbol de búsqueda binaria** (Binary Search Tree (BST)) es una estructura de datos arbórea en la que cada nodo contiene un par llave-valor y se cumplen las siguientes propiedades fundamentales:

1. Cada nodo tiene como máximo dos hijos: un hijo izquierdo y un hijo derecho.
2. Una llave solo puede aparecer una vez en un árbol. Las llaves no pueden ser nulas.
3. Para cualquier nodo, el valor de su llave es *mayor* que el valor de la llave de cualquier nodo en su subárbol **izquierdo**.
4. Para cualquier nodo, el valor de su llave es *menor* que el valor de la llave de cualquier nodo en su subárbol **derecho**.

Los BST son una implementación eficiente de la **tabla de símbolos ordenada**. En esencia, un BST permite mantener un conjunto de llaves ordenadas, facilitando búsquedas, inserciones y eliminaciones eficientes.

1.1.1 Aplicaciones Prácticas

Los BST tienen numerosas aplicaciones en el mundo real:

- Implementación de diccionarios y mapas
- Sistemas de caché
- Compiladores (tablas de símbolos)
- Bases de datos (índices)
- Algoritmos de compresión
- Sistemas de archivos

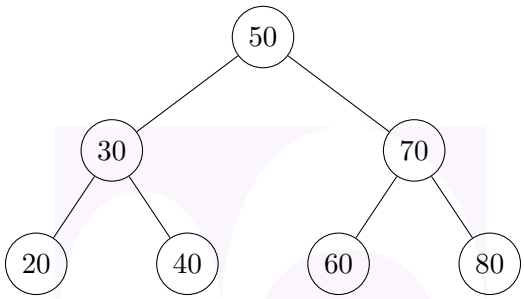
1.1.2 Construcción del BST

Desde una perspectiva de programación orientada a objetos, un BST se construye a partir de nodos. Cada nodo contiene:

- `Llave:` El valor que se utiliza para ordenar los nodos en el árbol.
- `Valor:` La información asociada a la llave.
- `Hijo Izquierdo:` Un puntero al subárbol izquierdo, que contiene nodos con llaves *menores* a la del nodo actual.
- `Hijo Derecho:` Un puntero al subárbol derecho, que contiene nodos con llaves *mayores* a la del nodo actual.

1.1.3 Ejemplo Visual

Consideremos un BST con las siguientes llaves: 50, 30, 70, 20, 40, 60, 80.



En este ejemplo:

- La raíz es 50
- Todos los nodos en el subárbol izquierdo son menores que 50
- Todos los nodos en el subárbol derecho son mayores que 50
- Cada subárbol también mantiene la propiedad de BST

1.2 Operaciones en el BST

Los BST soportan una variedad de operaciones esenciales. A continuación, se describen algunas de las más importantes, junto con su pseudocódigo y análisis de complejidad.

1.2.1 Get (Búsqueda)

La operación `get` busca un nodo con una llave específica en el árbol.

Algorithm 1 Pseudocódigo de la operación get

```
function GET(node, llave)
    if node = null then
        return null
    end if
    if llave < node.llave then
        return GET(node.izquierdo, llave)
    else if llave > node.llave then
        return GET(node.derecho, llave)
    else
        return node.valor
    end if
end function
```

Análisis de Complejidad: En el mejor de los casos, la complejidad es $O(1)$ (si la llave está en la raíz). En el peor de los casos, la complejidad es $O(h)$, donde h es la altura del árbol. En un árbol balanceado, h es $O(\log n)$, mientras que en un árbol no balanceado, h puede ser $O(n)$.

1.2.2 Put (Inserción)

La operación `put` inserta un nuevo nodo con una llave y valor dados en el árbol. Es esencial para construir y modificar el BST.

Algorithm 2 Pseudocódigo de la operación put

```
function PUT(node, llave, valor)
    if node = null then
        return NUEVONODO(llave, valor)
    end if
    if llave < node.llave then
        node.izquierdo ← PUT(node.izquierdo, llave, valor)
    else if llave > node.llave then
        node.derecho ← PUT(node.derecho, llave, valor)
    else
        node.valor ← valor
    end if
    return node
end function
```

Análisis de Complejidad: Similar a `get`, la complejidad es $O(h)$, donde h es la altura del árbol.

1.2.3 Delete (Eliminación)

La operación `delete` elimina un nodo con una llave específica del árbol. Esta operación es más compleja que `get` y `put`, ya que requiere reorganizar el árbol para mantener las propiedades del BST. Para su implementación se utiliza una regla sencilla como reemplazar el nodo a eliminar por su sucesor o predecesor.

Operación de borrado del mínimo La operación de borrado del mínimo en un BST elimina el nodo con la llave más pequeña del árbol. Dado que el nodo con la llave más pequeña se encuentra siempre en el extremo izquierdo del árbol (o subárbol), la operación consiste en descender recursivamente por el subárbol izquierdo hasta encontrar el nodo mínimo. Una vez encontrado, este nodo se elimina y se reemplaza por su hijo derecho (que puede ser nulo). Este proceso mantiene la propiedad de BST, ya que todos los nodos en el subárbol derecho del nodo eliminado son mayores que cualquier otro nodo en el árbol restante.

Análisis de Complejidad: La complejidad de esta operación es $O(h)$, donde h es la altura del árbol, ya que en el peor de los casos debe recorrerse la altura completa del árbol para encontrar el mínimo.

Operación de borrado de nodos en general El borrado de un nodo en general es una operación más compleja que el borrado del mínimo, ya que requiere considerar varios casos para mantener las propiedades del BST. El proceso general consta de los siguientes pasos:

1. **Buscar el nodo a borrar:** Se busca el nodo con la llave que se desea eliminar. Sea t la referencia a este nodo.

Algorithm 3 Pseudocódigo de la operación deleteMin

```
function DeleteMin(node)
    if node = null then
        return null
    end if
    if node.izquierdo = null then
        return node.derecho
    end if
    node.izquierdo ← DeleteMin(node.izquierdo)
    return node
end function
```

2. **Caso 1: El nodo a borrar no tiene hijos:** Simplemente se elimina el nodo. Esto se reduce a establecer el puntero del padre a nulo.

3. **Caso 2: El nodo a borrar tiene un solo hijo:** Se reemplaza el nodo por su único hijo. El hijo puede ser izquierdo o derecho.

4. **Caso 3: El nodo a borrar tiene dos hijos:** Se encuentra el sucesor de t, que es el nodo con la llave más pequeña en el subárbol derecho de t. Sea x el sucesor de t, es decir, $x = \textsc{Min}(t.derecho)$. El nodo x reemplazará a t en el árbol. Luego, se actualizan los hijos de x:

   - El hijo derecho de x se obtiene eliminando el mínimo del subárbol derecho de t: $x.derecho = \textsc{DeleteMin}(t.derecho)$.
   - El hijo izquierdo de x es el hijo izquierdo de t: $x.izquierdo = t.izquierdo$.

5. **Actualizar el tamaño:** Se actualiza el tamaño de los nodos en el camino de retorno de las llamadas recursivas.

Análisis de Complejidad: La complejidad de esta operación es $O(h)$, donde h es la altura del árbol. En el peor de los casos, la búsqueda del nodo a eliminar y la búsqueda de su sucesor (o predecesor) pueden requerir recorrer la altura completa del árbol.

1.2.4 Otras operaciones

Además de las operaciones básicas, los BST, como tablas de símbolos ordenadas soportan otras operaciones:

- **Mínimo:** Encuentra el nodo con la llave mínima.
- **Máximo:** Encuentra el nodo con la llave máxima.
- **Piso (Floor):** Encuentra el nodo con la llave más grande menor o igual a una llave dada.
- **Techo (Ceiling):** Encuentra el nodo con la llave más pequeña mayor o igual a una llave dada.
- **Rango (Rank):** Determina cuántas llaves en el árbol son menores que una llave dada.
- **Seleccionar (Select):** Encuentra la llave que tiene un rango específico.

La complejidad de estas operaciones también es $O(h)$.

Algorithm 4 Pseudocódigo de la operación delete

```
function DELETE(node, llave)
    if node = null then
        return null
    end if
    if llave < node.llave then
        node.izquierdo ← DELETE(node.izquierdo, llave)
    else if llave > node.llave then
        node.derecho ← DELETE(node.derecho, llave)
    else
        if node.derecho = null then
            return node.izquierdo
        end if
        if node.izquierdo = null then
            return node.derecho
        end if
        t ← node
        node ← MIN(t.derecho)
        node.derecho ← DELETEMIN(t.derecho)
        node.izquierdo ← t.izquierdo
    end if
    return node
end function
```

1.3 Recorrido de Árboles

El recorrido de un árbol implica visitar cada nodo del árbol exactamente una vez. Hay varias formas de recorrer un BST, cada una con sus propias características y aplicaciones.

1.3.1 Preorden (Raíz-Izquierda-Derecha)

En el recorrido en preorden:

1. Se visita el nodo raíz
2. Se recorre el subárbol izquierdo en preorden
3. Se recorre el subárbol derecho en preorden

Aplicaciones:

- Creación de una copia del árbol
- Serialización del árbol
- Evaluación de expresiones prefijas

1.3.2 Inorden (Izquierda-Raíz-Derecha)

En el recorrido inorden:

1. Se recorre el subárbol izquierdo en inorden
2. Se visita el nodo raíz
3. Se recorre el subárbol derecho en inorden

Aplicaciones:

- Obtener elementos en orden
- Validación de BST
- Búsquedas por rango

1.3.3 Postorden (Izquierda-Derecha-Raíz)

En el recorrido postorden:

1. Se recorre el subárbol izquierdo en postorden
2. Se recorre el subárbol derecho en postorden
3. Se visita el nodo raíz

Aplicaciones:

- Eliminación de árboles
- Evaluación de expresiones postfijas
- Cálculo de alturas y tamaños

1.4 Consideraciones de Implementación

1.4.1 Mejores Prácticas

- Mantener el árbol balanceado para optimizar el rendimiento
- Implementar validación de propiedades del BST
- Manejar casos especiales (árbol vacío, nodos nulos)
- Implementar iteradores para recorridos eficientes

1.4.2 Errores Comunes

- No verificar propiedades del BST durante inserciones
- Olvidar actualizar punteros durante eliminaciones
- No manejar casos de árbol degenerado
- Ignorar el balance del árbol

1.5 Optimizaciones

1.5.1 Árboles Balanceados

Para mantener un rendimiento óptimo, es crucial mantener el árbol balanceado. Algunas implementaciones comunes incluyen:

- Árboles AVL (ver Sección 5)
- Árboles Rojo-Negro (ver Sección 4)
- Árboles 2-3 (ver Sección 3)
- Árboles B y B+ (ver Sección 5)

1.5.2 Casos de Uso y Selección

La elección entre diferentes tipos de árboles balanceados depende de:

- Frecuencia de operaciones de lectura vs. escritura
- Requisitos de memoria
- Necesidad de operaciones concurrentes
- Patrones de acceso a los datos

2 Ejercicios y Práctica

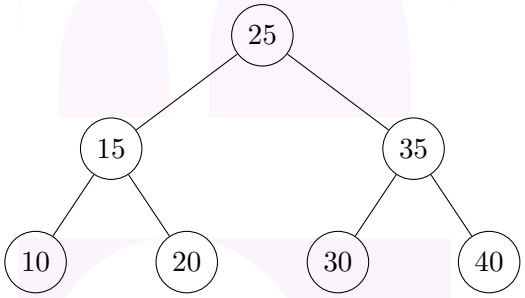
2.1 Ejercicios Teóricos

1. **Propiedades del BST**
   (a) Demuestra que en un BST, el recorrido inorden siempre produce una secuencia ordenada de llaves.
   (b) ¿Por qué es importante que las llaves en un BST sean únicas?
   (c) Explica por qué un árbol binario completo no necesariamente es un BST.
2. **Análisis de Complejidad**
   (a) ¿Cuál es la complejidad de buscar un elemento en un BST degenerado (que se asemeja a una lista enlazada)?
   (b) Explica por qué la complejidad de las operaciones básicas en un BST balanceado es $O(\log n)$.
   (c) ¿Cuál es la complejidad de recorrer todo el árbol en cualquiera de los tres órdenes (preorden, inorden, postorden)?
3. **Operaciones**
   (a) Describe el proceso de eliminar un nodo con dos hijos en un BST.
   (b) ¿Por qué es importante mantener el árbol balanceado?
   (c) Explica la diferencia entre el sucesor y el predecesor de un nodo en un BST.

2.2 Ejercicios Prácticos

2.2.1 Construcción de BST

1. Construye un BST insertando los siguientes números en orden: 45, 30, 60, 20, 40, 50, 70.
   (a) Dibuja el árbol resultante
   (b) Realiza un recorrido inorden y escribe la secuencia resultante
   (c) Encuentra el sucesor de 45
   (d) Encuentra el predecesor de 60
2. Dado el siguiente BST:



   (a) Elimina el nodo 15 y dibuja el árbol resultante
   (b) Elimina el nodo 25 y dibuja el árbol resultante
   (c) Inserta el nodo 22 y dibuja el árbol resultante

2.2.2 Implementación

1. Implementa las siguientes operaciones en tu lenguaje de programación preferido:
   (a) Inserción de un nodo
   (b) Búsqueda de un nodo
   (c) Eliminación de un nodo
   (d) Recorrido inorden
2. Extiende la implementación anterior para incluir:
   (a) Cálculo de la altura del árbol
   (b) Verificación de si el árbol está balanceado
   (c) Búsqueda por rango
   (d) Encontrar el k-ésimo elemento más pequeño

2.3 Problemas de Desafío

1. **Validación de BST**
   (a) Implementa una función que verifique si un árbol binario dado es un BST válido.
   (b) Considera los casos especiales: árbol vacío, árbol con un solo nodo, árbol degenerado.

   (c) Optimiza tu solución para que sea eficiente en términos de tiempo y espacio.

2. **Balanceo de BST**
   (a) Implementa una función que balancee un BST dado.
   (b) Considera diferentes estrategias de balanceo (AVL, Rojo-Negro).
   (c) Analiza la complejidad de tu solución.

3. **Operaciones Avanzadas**
   (a) Implementa una operación para encontrar el ancestro común más bajo (LCA) de dos nodos.
   (b) Implementa una operación para encontrar la suma de todos los nodos en un rango dado.
   (c) Implementa una operación para convertir un BST en una lista doblemente enlazada ordenada.

2.4 Soluciones y Pistas

2.4.1 Pistas para los Ejercicios Teóricos

1. Para demostrar que el recorrido inorden produce una secuencia ordenada, considera la propiedad de BST que establece que todos los elementos en el subárbol izquierdo son menores que la raíz.

2. La complejidad en un árbol degenerado es $O(n)$ porque el árbol se convierte esencialmente en una lista enlazada.

3. Al eliminar un nodo con dos hijos, considera usar el sucesor o predecesor para mantener las propiedades del BST.

2.4.2 Pistas para los Ejercicios Prácticos

1. Al construir el BST, recuerda que cada nuevo nodo debe mantener la propiedad de que todos los elementos a su izquierda son menores y todos los elementos a su derecha son mayores.

2. Para la implementación, considera usar una estructura de nodo con punteros a los hijos izquierdo y derecho.

3. Al implementar la eliminación, considera los tres casos: nodo sin hijos, nodo con un hijo, y nodo con dos hijos.

2.4.3 Pistas para los Problemas de Desafío

1. Para la validación de BST, considera usar un recorrido inorden y verificar que cada elemento sea mayor que el anterior.

2. Para el balanceo, considera usar rotaciones y mantener información de balance en cada nodo.

3. Para el LCA, considera que el ancestro común más bajo debe estar entre los dos nodos en el recorrido inorden.

3 Árboles de búsqueda 2-3

3.1 Definición

Los árboles de búsqueda 2-3 son una estructura de datos arbórea diseñada para mantener el árbol balanceado, garantizando un rendimiento de búsqueda en el peor de los casos de $O(\log n)$, donde n es el número de elementos en el árbol. A diferencia de los árboles binarios de búsqueda, los árboles 2-3 permiten que cada nodo tenga dos o tres hijos.

Definición formal:

Un árbol 2-3 es un árbol que satisface las siguientes propiedades:

- Se compone de dos tipos de nodos: 2-nodes y 3-nodes.
- Los 2-nodes tienen 2 hijos y un par llave-valor.
- Los 3-nodes tienen 3 hijos y dos pares llave-valor.
- Todos los nodos hoja están al mismo nivel, lo que garantiza que el árbol esté balanceado.
- Los valores en los nodos se mantienen en orden de búsqueda, similar a los árboles binarios de búsqueda.

3.2 Estructura Visual

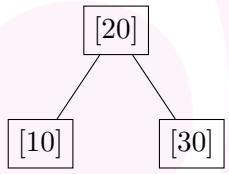


Figure 1: Ejemplo de un 2-node con dos hijos. El nodo contiene la llave 20, con llaves menores a la izquierda y mayores a la derecha.

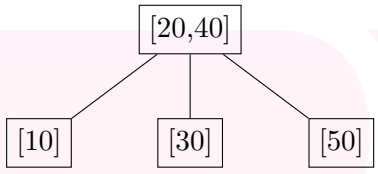


Figure 2: Ejemplo de un 3-node con tres hijos. El nodo contiene las llaves 20 y 40, con tres rangos de valores.

3.3 Búsqueda en árboles 2-3

El algoritmo de búsqueda en un árbol 2-3 es una extensión de la búsqueda en un árbol binario de búsqueda. Comienza en la raíz y compara el valor buscado con los valores almacenados en el nodo actual.

- Si la llave buscada es igual a alguno de los valores en el nodo, la búsqueda se completa.

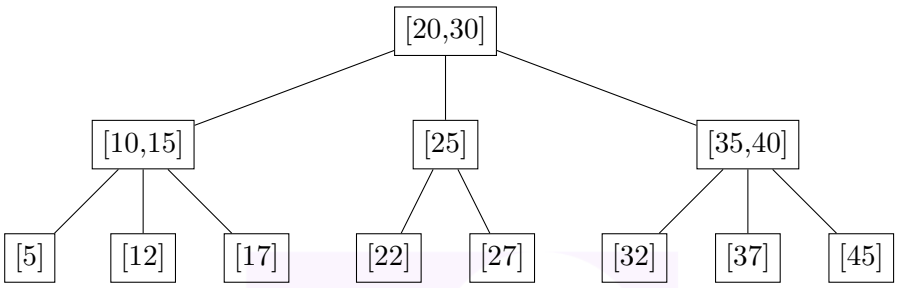


Figure 3: Ejemplo de un árbol 2-3 completo

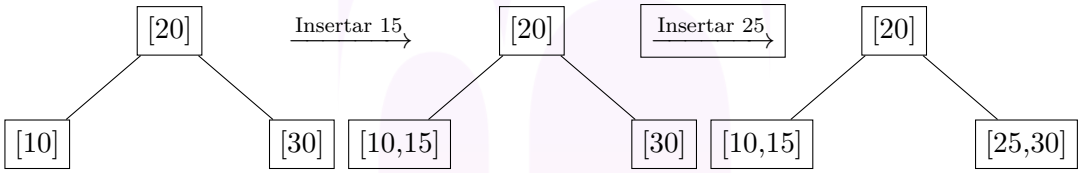


Figure 4: Pasos de inserción en un árbol 2-3. Las inserciones se realizan en los nodos hoja.

- Si la llave buscada es menor que el valor más pequeño en el nodo, la búsqueda continúa en el hijo más a la izquierda.
- Si la llave buscada está entre las llaves en el nodo, la búsqueda continúa en el hijo central.
- Si la llave buscada es mayor que la llave mayor, la búsqueda continúa en el hijo más a la derecha.

La búsqueda continúa hasta que se encuentra el valor o se llega a un nodo hoja.

3.4 Inserción en árboles 2-3

La inserción en un árbol 2-3 también debe mantener la propiedad de equilibrio del árbol. El proceso general es el siguiente:

1. **Buscar la posición de inserción:** Se realiza una búsqueda para encontrar el nodo hoja donde se debe insertar el nuevo valor.
2. **Insertar en el nodo hoja:**
   - Si el nodo hoja tiene espacio (es decir, es un nodo 2), el nuevo valor se inserta en el nodo, manteniendo el orden.
   - Si el nodo hoja está lleno (es decir, es un nodo 3), se divide el nodo en dos nodos, y el valor medio se promueve al nodo padre.
3. **Propagar la división (si es necesario):** Si la promoción del valor medio hace que el nodo padre se llene, el proceso de división se repite en el nodo padre. Esto puede propagarse hasta la raíz del árbol. Si la raíz se divide, se crea una nueva raíz, y la altura del árbol aumenta en uno.

3.5 Eliminación en árboles 2-3

La eliminación en árboles 2-3 es más compleja que la inserción y requiere varios casos especiales:

1. **Eliminación en un nodo hoja:**
   - Si el nodo es un 3-node, simplemente se elimina la llave.
   - Si el nodo es un 2-node, se debe realizar una redistribución o fusión con nodos hermanos.
2. **Eliminación en un nodo interno:**
   - Se reemplaza la llave a eliminar con su predecesor o sucesor.
   - Se elimina el predecesor o sucesor de su posición original.
3. **Redistribución y fusión:**
   - Si un nodo se queda sin llaves, se intenta redistribuir llaves desde nodos hermanos.
   - Si la redistribución no es posible, se fusiona con un nodo hermano.

3.6 Ventajas y Desventajas

Ventajas:

- Garantiza un rendimiento de $O(\log n)$ para todas las operaciones.
- Mantiene el árbol balanceado automáticamente.
- No requiere rotaciones, pero utiliza operaciones de división y fusión de nodos para mantener el balance.

Desventajas:

- Implementación más compleja que los árboles binarios de búsqueda.
- Mayor overhead de memoria por nodo.
- Operaciones de inserción y eliminación más complejas.

3.7 Pseudocódigo de Operaciones Básicas

3.8 Complejidad Algorítmica

Las operaciones de búsqueda, inserción y eliminación en árboles 2-3 tienen una complejidad algorítmica de $O(\log n)$ en el peor de los casos, debido a la propiedad de equilibrio del árbol. Esto los convierte en una opción eficiente para mantener conjuntos de datos ordenados donde el rendimiento de la búsqueda es crítico.

Algorithm 5 Búsqueda en Árbol 2-3

```
function BUSCAR(nodo, llave)
    if nodo es nulo then
        return No encontrado
    end if
    if llave está en nodo then
        return Encontrado
    else if nodo es 2-node then
        if llave ¡ nodo.llave then
            return Buscar(nodo.izquierdo, llave)
        else
            return Buscar(nodo.derecho, llave)
        end if
    else
        if llave ¡ nodo.llave1 then
            return Buscar(nodo.izquierdo, llave)
        else if llave ¡ nodo.llave2 then
            return Buscar(nodo.central, llave)
        else
            return Buscar(nodo.derecho, llave)
        end if
    end if
end function
```

3.9 Relación con Árboles Rojo-Negros

Los árboles 2-3 son conceptualmente más simples que los árboles rojo-negros, pero su implementación directa es más compleja. Por esta razón, en la práctica se suelen implementar árboles rojo-negros, que son una representación binaria de los árboles 2-3. Los árboles rojo-negros mantienen las mismas propiedades de equilibrio pero con una implementación más eficiente en términos de memoria y operaciones.

Si bien los árboles 2-3 son una excelente herramienta conceptual para entender cómo se mantiene el balance en un árbol de búsqueda, su implementación directa es poco común. La principal dificultad radica en la necesidad de manejar dos tipos de nodos diferentes (2-nodos y 3-nodos), lo que complica significativamente el código para las operaciones de inserción y eliminación. Cada operación debe considerar múltiples casos para la división, fusión y redistribución de llaves.

En contraste, los árboles rojo-negros (que se presentan en la siguiente sección) representan la misma estructura de un árbol 2-3 utilizando únicamente nodos de un solo tipo (nodos de árbol binario) con un bit de color adicional. Esta representación, aunque requiere operaciones de ”rotación” y ”cambio de color” para mantener el balance, resulta en un código más uniforme y, a menudo, más simple de implementar y mantener. Por lo tanto, los árboles rojo-negros son la elección pragmática para implementar árboles de búsqueda balanceados en la práctica.

4 Árboles Rojo-Negros

4.1 Definición

Los árboles rojo-negros son una clase de árboles de búsqueda auto-balanceables. Además de los atributos típicos de un árbol binario de búsqueda, cada nodo en un árbol rojo-negro tiene un atributo de color, que puede ser rojo o negro. Estos colores se utilizan para asegurar que el árbol permanezca balanceado durante las inserciones y eliminaciones.

Definición formal:
Un árbol rojo-negro es un árbol binario de búsqueda que satisface las siguientes propiedades:

1. Cada nodo es rojo o negro.
2. La raíz es negra.
3. Si un nodo es rojo, entonces sus dos hijos son negros.
4. Todos los caminos desde cualquier nodo a todas sus hojas descendientes contienen el mismo número de nodos negros.

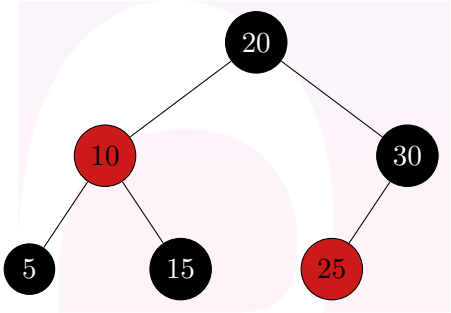


Figure 5: Ejemplo de un árbol rojo-negro válido. Los nodos negros se muestran con fondo negro y texto blanco, los rojos con un fondo rojo claro.

La propiedad clave de los árboles rojo-negros es que garantizan que la altura del árbol sea a lo sumo $2\lg(n+1)$, donde n es el número de nodos internos (no hoja) en el árbol. Esta cota se deriva del hecho de que cada camino desde la raíz hasta una hoja contiene el mismo número de nodos negros (altura negra), y puede haber como máximo un nodo rojo entre cada par de nodos negros consecutivos. Esto asegura que las operaciones de búsqueda, inserción y eliminación puedan realizarse en tiempo $O(\lg n)$.

4.2 Relación con los Árboles 2-3

Existe una estrecha relación entre los árboles rojo-negros y los árboles 2-3. Un árbol rojo-negro puede ser visto como una representación binaria de un árbol 2-3. En esta correspondencia, los enlaces rojos ”pegan” nodos para formar el equivalente de un 3-nodo.

- Un **2-nodo** se representa como un nodo negro con dos hijos negros.
- Un **3-nodo** se representa como un nodo negro con un hijo rojo. En las implementaciones de **árboles rojo-negros zurdos (Left-Leaning Red-Black Trees)**, el hijo rojo siempre es el izquierdo.

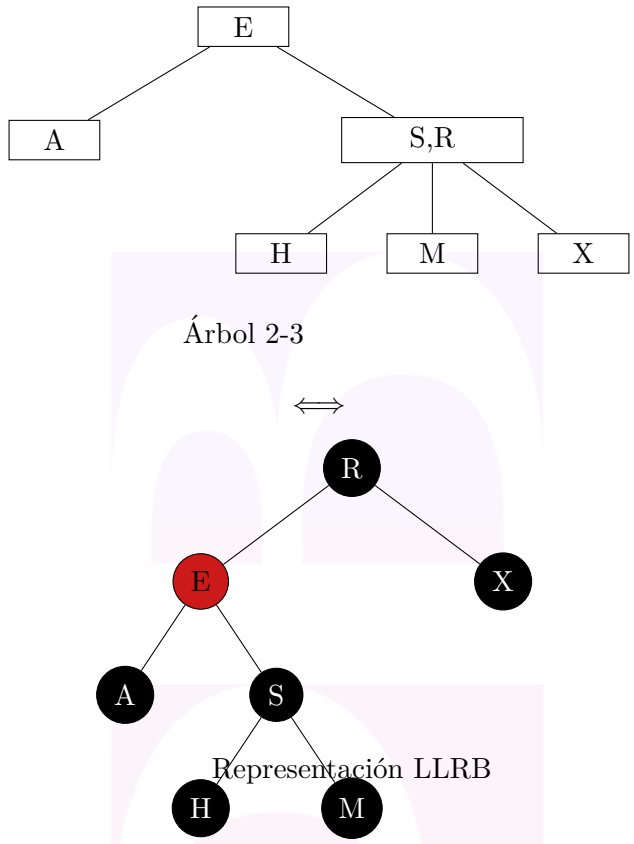


Figure 6: Transformación de un árbol 2-3 a su representación en árbol rojo-negro zurdo (LLRB). El 3-nodo (S,R) se representa como un nodo negro (R) con un hijo izquierdo rojo (S).

Esta correspondencia asegura que la altura de un árbol rojo-negro sea logarítmica, ya que simula la estructura balanceada de un árbol 2-3.

4.3 Implementación del Árbol Rojo-Negro

La implementación de un árbol rojo-negro implica mantener la estructura del árbol binario de búsqueda y las propiedades de coloración. Las operaciones clave son la inserción y la eliminación, que deben preservar las propiedades del árbol rojo-negro. Esto se logra mediante rotaciones y recoloreos. La implementación que se presenta aquí corresponde a un **árbol rojo-negro zurdo (LLRB)**, que simplifica el código al reducir el número de casos a considerar.

4.3.1 Representación del Color

El color de un nodo se representa generalmente con un valor booleano: `True` para rojo y `False` para negro. En la implementación, es común usar constantes para mayor claridad:

```
private static final boolean RED = true;
private static final boolean BLACK = false;
```

4.3.2 Rotaciones

Las rotaciones son operaciones que modifican la estructura local del árbol sin afectar el orden de las llaves. Hay dos tipos de rotaciones:

- **Rotación a la izquierda:** Permite mover el hijo derecho hacia la posición de su padre.
- **Rotación a la derecha:** Permite mover el hijo izquierdo hacia la posición de su padre.

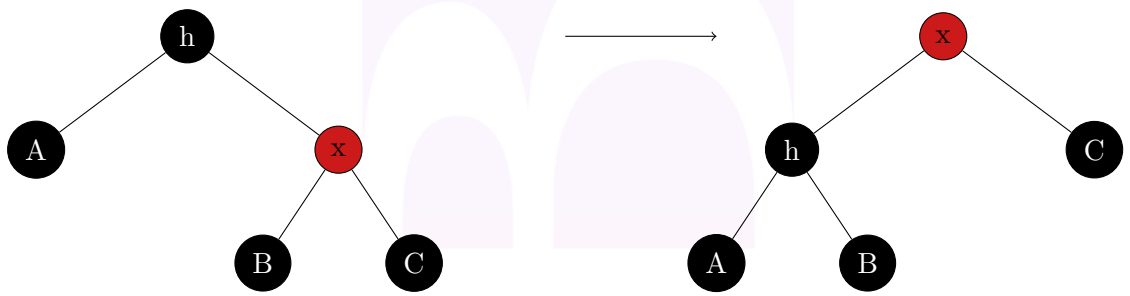


Figure 7: Ejemplo de rotación a la izquierda. La operación mueve el enlace rojo de la derecha a la izquierda.

Algorithm 6 Pseudocódigo de Rotación a la Izquierda

```
function LeftRotate(Node h)
    x ← h.right
    h.right ← x.left
    x.left ← h
    x.color ← h.color
    h.color ← RED
    x.n ← h.n
    h.n ← size(h.left) + size(h.right) + 1
    return x
end function
```

Algorithm 7 Pseudocódigo de Rotación a la Derecha

```
function RightRotate(Node h)
    x ← h.left
    h.left ← x.right
    x.right ← h
    x.color ← h.color
    h.color ← RED
    x.n ← h.n
    h.n ← size(h.left) + size(h.right) + 1
    return x
end function
```

4.3.3 Inserción

La inserción de un nuevo nodo en un árbol rojo-negro sigue estos pasos:

1. Insertar el nuevo nodo como en un BST normal, coloreándolo de rojo.
2. Subir por el árbol corrigiendo los enlaces rojos que no cumplan la restricción zurda o que formen 4-nodos temporales.

Ejemplo de inserción: Al insertar la secuencia S E A R C H en un LLRB vacío, se realizan las siguientes operaciones para mantener el balance.

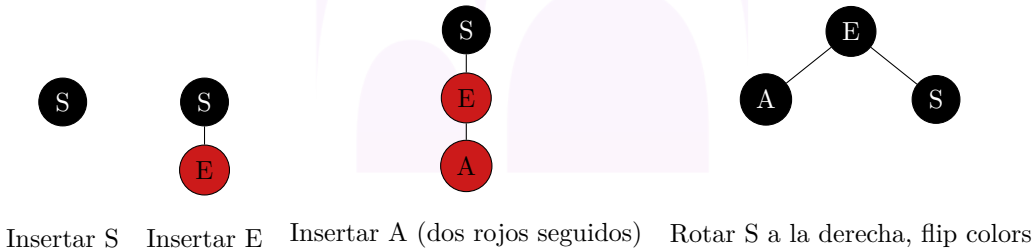


Figure 8: Ejemplo de inserción en un LLRB (Parte 1).

4.3.4 Eliminación

La eliminación en un árbol rojo-negro es más compleja que la inserción. El proceso sigue estos pasos:

1. Realizar la eliminación como en un BST normal.
2. Si el nodo eliminado era rojo, no se necesitan correcciones.
3. Si el nodo eliminado era negro, se necesitan correcciones para mantener las propiedades:
   - Si el hermano es rojo, realizar una rotación y recolorear.
   - Si el hermano es negro con dos hijos negros, recolorear.
   - Si el hermano es negro con al menos un hijo rojo, realizar rotaciones y recoloreos.

Nota: La implementación completa de la eliminación en árboles rojo-negros es considerablemente compleja, con múltiples casos a considerar. Para un tratamiento detallado de la eliminación, se recomienda consultar textos especializados como *Introduction to Algorithms* de Cormen et al., o recursos en línea como las implementaciones en las librerías estándar de Java (`TreeMap`) o C++ (`std::map`).

4.4 Eficiencia de Operaciones en Árboles Rojo-Negros

Las operaciones de búsqueda, inserción y eliminación en árboles rojo-negros tienen una complejidad temporal de $O(\log n)$ en el peor caso. Esto se debe a que la altura del árbol se mantiene logarítmica mediante las propiedades de coloración y las operaciones de reestructuración (rotaciones y recoloreos).

Algorithm 8 Pseudocódigo de la operación put

```
function PUT(llave, valor)
    root ← PUT(root, llave, valor)
    root.color ← BLACK
end function
function PUT(Node h, llave, valor)
    if h == null then
        return new NODE(key, val, RED, 1);
    end if
    cmp ← key.compareTo(h.key)
    if cmp < 0 then
        h.left ← PUT(h.left, llave, valor);
    else if cmp > 0 then
        h.right ← PUT(h.right, llave, valor);
    else
        h.val ← valor;
    end if
    if isRed(h.right) and !isRed(h.left) then
        h =ROTATELEFT(h)
    end if
    if isRed(h.left) and isRed(h.left.left) then
        h =ROTATERIGHT(h)
    end if
    if isRed(h.left) and isRed(h.right) then
        FLIPCOLORS(h)
    end if
    h.N = size(h.left) + size(h.right) + 1
    return h
end function
```

4.5 Consideraciones Prácticas

- **Ventajas:**
  - Garantiza operaciones en tiempo $O(\log n)$.
  - Menos rotaciones que los árboles AVL en promedio.
  - Buena para implementaciones en memoria.
- **Desventajas:**
  - Mayor complejidad de implementación que un BST simple.
  - Overhead de memoria por el bit de color.
- **Casos de Uso:**
  - Implementación de mapas y conjuntos ordenados en librerías estándar (ej. ‘std::map‘ en C++, ‘TreeMap‘ en Java).
  - Cuando se necesita garantizar tiempo $O(\log n)$ para búsquedas, inserciones y eliminaciones.
  - Cuando la memoria no es una restricción crítica.

4.6 Ejercicios Recomendados

1. Implementar las operaciones básicas de un árbol rojo-negro.
2. Demostrar que la altura es a lo sumo $2\lg(n+1)$.
3. Analizar los casos de rotación y recoloreo en la inserción.
4. Comparar el rendimiento con otros árboles balanceados.

5 Otros tipos de árboles de búsqueda

Esta sección presenta una breve introducción a diferentes tipos de árboles de búsqueda balanceados. Cada tipo de árbol tiene sus propias características y casos de uso específicos. Se incluyen enlaces a recursos externos para un estudio más profundo de cada estructura.

5.1 Árboles AVL

Los árboles AVL, nombrados en honor a sus inventores Adelson-Velsky y Landis (1962), son árboles de búsqueda binarios auto-balanceables donde las alturas de los subárboles izquierdo y derecho de cualquier nodo difieren como máximo en uno. Esta propiedad de **balance de altura** asegura que el árbol permanezca balanceado después de cada operación.

5.1.1 Características Principales

- **Factor de Balance:** Para cada nodo, el factor de balance se define como $|altura(subárbol_izquierdo) - altura(subárbol_derecho)| \leq 1$.
- **Altura Garantizada:** La altura del árbol está acotada por $h \leq 1.44 \log_2(n+2)$, donde n es el número de nodos.
- **Operaciones de Rebalanceo:** Utiliza cuatro tipos de rotaciones para mantener el balance: rotación simple izquierda, rotación simple derecha, rotación doble izquierda-derecha, y rotación doble derecha-izquierda.
- **Complejidad:** Todas las operaciones básicas (búsqueda, inserción, eliminación) tienen complejidad $O(\log n)$ garantizada en el peor caso.

5.1.2 Ventajas y Desventajas

Ventajas:

- Balance más estricto que los árboles rojo-negros, lo que resulta en búsquedas ligeramente más rápidas.
- Búsquedas más predecibles en términos de rendimiento.
- Ideal para aplicaciones con muchas más operaciones de lectura que de escritura.

Desventajas:

- Requiere más rotaciones durante inserciones y eliminaciones que los árboles rojo-negros (aproximadamente 1.5 veces más).
- Overhead de almacenamiento adicional para mantener el factor de balance en cada nodo.
- Operaciones de inserción y eliminación ligeramente más lentas que en árboles rojo-negros.

5.1.3 Casos de Uso

Los árboles AVL son particularmente útiles en:

- Bases de datos en memoria donde las búsquedas son mucho más frecuentes que las actualizaciones.
- Sistemas de tiempo real donde se necesita un tiempo de respuesta predecible.
- Aplicaciones que requieren acceso frecuente a elementos ordenados.
- AVL Trees @ w3schools
- Wikipedia
- AVL Tree Data Structure @ Geeks for Geeks
- AVL Tree @ Programiz
- AVL Tree Visualization

5.2 Árboles B (B-Trees)

Los árboles B, inventados por Rudolf Bayer y Edward McCreight en 1972, son estructuras de datos de árbol auto-balanceables diseñadas específicamente para sistemas de almacenamiento que leen y escriben bloques de datos relativamente grandes, como bases de datos y sistemas de archivos. A diferencia de los árboles binarios, los árboles B pueden tener muchos más de dos hijos por nodo, lo que los hace ideales para minimizar el número de accesos al disco.

5.2.1 Características Principales

Un árbol B de orden m satisface las siguientes propiedades:

- **Factor de Ramificación:** Cada nodo interno (excepto la raíz) tiene entre $\lceil m/2 \rceil$ y m hijos.
- **Llaves por Nodo:** Cada nodo interno contiene entre $\lceil m/2 \rceil - 1$ y $m - 1$ llaves.
- **Balance Perfecto:** Todas las hojas están al mismo nivel, garantizando balance perfecto.
- **Orden en Llaves:** Las llaves dentro de cada nodo están ordenadas, y las llaves en los subárboles mantienen la propiedad de orden del BST.
- **Altura Logarítmica:** La altura es $O(\log_m n)$, donde n es el número de llaves y m es el orden del árbol.

5.2.2 Ventajas y Desventajas

Ventajas:

- **Optimización para Disco:** Minimiza el número de accesos al disco al almacenar múltiples llaves por nodo.
- **Alto Factor de Ramificación:** Reduce la altura del árbol significativamente, especialmente para valores grandes de m.

- **Balance Garantizado:** Todas las hojas están al mismo nivel, garantizando rendimiento predecible.
- **Eficiente para Rangos:** Excelente para consultas de rango debido a la localidad de las llaves.

Desventajas:

- **Complejidad de Implementación:** Más complejo de implementar que los árboles binarios de búsqueda.
- **Overhead de Espacio:** Los nodos pueden no estar completamente llenos, desperdiciando espacio.
- **No Óptimo para Memoria:** El overhead es significativo para estructuras completamente en memoria.

5.2.3 Casos de Uso

Los árboles B son la estructura estándar en:

- **Sistemas de Bases de Datos:** Utilizados para índices en sistemas como MySQL, PostgreSQL, y Oracle.
- **Sistemas de Archivos:** Implementados en sistemas de archivos como NTFS, HFS+, ext4, y Btrfs.
- **Almacenamiento en Disco:** Cualquier aplicación donde los datos residen principalmente en disco y el acceso al disco es costoso.
- Introduction of B-Tree @ Geeks for Geeks
- Wikipedia
- B-Tree Visualization

5.3 Árboles B+ (B+ Trees)

Los árboles B+ son una variante de los árboles B con una diferencia arquitectónica fundamental: **todos los datos se almacenan exclusivamente en los nodos hoja**, mientras que los nodos internos solo contienen llaves que actúan como índices para guiar las búsquedas. Además, los nodos hoja están enlazados entre sí, formando una lista enlazada ordenada que facilita enormemente las operaciones de rango y el acceso secuencial.

5.3.1 Características Principales

- **Datos en Hojas:** Todos los pares llave-valor residen únicamente en los nodos hoja; los nodos internos solo contienen copias de llaves para navegación.
- **Hojas Enlazadas:** Los nodos hoja forman una lista doblemente enlazada, permitiendo recorridos secuenciales eficientes sin necesidad de volver a la raíz.
- **Mayor Densidad de Llaves:** Los nodos internos pueden almacenar más llaves que en un árbol B tradicional, ya que no necesitan almacenar valores.

- **Altura Consistente:** Al igual que los árboles B, todas las hojas están al mismo nivel.
- **Operaciones de Rango Optimizadas:** Las consultas de rango son extremadamente eficientes gracias al enlazamiento de hojas.

5.3.2 Diferencias con Árboles B

Característica	Árbol B	Árbol B+
Datos en nodos internos	Sí	No (solo índices)
Datos en hojas	Algunos	Todos
Hojas enlazadas	No	Sí
Consultas de rango	Buenas	Excelentes
Densidad de llaves internas	Menor	Mayor
Acceso secuencial	Requiere recorrido	Muy eficiente

5.3.3 Ventajas y Desventajas

Ventajas:

- **Consultas de Rango Superiores:** El enlazamiento de hojas permite recorrer rangos sin volver a los nodos internos.
- **Mayor Factor de Ramificación:** Los nodos internos pueden almacenar más llaves, reduciendo aún más la altura.
- **Acceso Secuencial Eficiente:** Ideal para operaciones de escaneo completo o parcial.
- **Mejor para Bases de Datos:** Optimizado para las operaciones más comunes en sistemas de bases de datos.

Desventajas:

- **Duplicación de Llaves:** Las llaves aparecen tanto en nodos internos como en hojas, aumentando el uso de espacio.
- **Complejidad de Implementación:** Ligeramente más complejo que los árboles B debido al manejo de enlaces entre hojas.
- **Búsquedas Puntuales:** Siempre deben llegar hasta las hojas, mientras que en árboles B pueden terminar antes.

5.3.4 Casos de Uso

Los árboles B+ son predominantes en:

- **Motores de Bases de Datos Relacionales:** Implementación estándar para índices en PostgreSQL, MySQL (InnoDB), SQL Server.
- **Sistemas de Archivos Modernos:** XFS, ReiserFS utilizan variantes de B+ trees.
- **Almacenamiento de Series Temporales:** Donde las consultas de rango por tiempo son frecuentes.
- **Data Warehouses:** Para consultas analíticas que requieren escaneos de rangos grandes.

5.3.5 Referencias Adicionales

Para profundizar en los árboles B+, consultar:

- Introduction of B+ Tree @ Geeks for Geeks
- The Difference Between B-trees and B+trees @ Baeldung
- Wikipedia
- B+ Tree Visualization

5.4 Comparación de Estructuras de Árboles de Búsqueda

La siguiente tabla resume las características principales de las diferentes estructuras de árboles de búsqueda presentadas en este capítulo:

Estructura	**Altura Máxima**	**Balance**	**Complejidad de Implementación**	**Mejor Caso de Uso**
BST Simple	$O(n)$	No garantizado	Baja	Datos aleatorios, prototipos
Árbol 2-3	$O(\log n)$	Perfecto	Alta	Estudio teórico
Árbol Rojo-Negro	$O(\log n)$	Bueno	Media-Alta	Mapas/conjuntos en memoria con inserciones frecuentes
Árbol AVL	$O(\log n)$	Muy estricto	Media-Alta	Búsquedas frecuentes, pocas inserciones
Árbol B	$O(\log_m n)$	Perfecto	Alta	Índices en disco, sistemas de archivos
Árbol B+	$O(\log_m n)$	Perfecto	Alta	Bases de datos, consultas de rango

5.4.1 Guía de Selección

Usa BST simple cuando:

- Los datos se insertan en orden aleatorio
- Es un prototipo o aplicación de aprendizaje
- No hay requisitos estrictos de rendimiento en el peor caso

Usa árboles Rojo-Negro cuando:

- Necesitas operaciones balanceadas en memoria
- Las inserciones y eliminaciones son frecuentes

- Quieres una implementación relativamente simple de árbol balanceado
- Es el estándar en `TreeMap` (Java), `std::map` (C++)

Usa árboles AVL cuando:

- Las búsquedas son mucho más frecuentes que las modificaciones
- Necesitas el mejor tiempo de búsqueda posible
- El tiempo de respuesta predecible es crítico

Usa árboles B o B+ cuando:

- Los datos residen principalmente en disco
- Estás implementando un sistema de base de datos o sistema de archivos
- Las consultas de rango son frecuentes (prefiere B+ en este caso)
- El factor de ramificación alto puede reducir la altura significativamente

5.4.2 Resumen de Rendimiento

Todas las estructuras balanceadas (2-3, Rojo-Negro, AVL, B, B+) garantizan $O(\log n)$ para operaciones básicas, pero difieren en:

- **Constantes multiplicativas:** AVL tiene búsquedas ligeramente más rápidas que RB, pero inserciones más lentas.
- **Número de rotaciones:** RB requiere menos rotaciones que AVL en promedio.
- **Uso de memoria:** BST y RB tienen menor overhead por nodo; árboles B tienen nodos más grandes pero menos nodos totales.
- **Operaciones de rango:** B+ es superior para rangos; BST/AVL/RB requieren recorridos en orden.
- **Acceso al disco:** B y B+ minimizan accesos al disco; los demás son para estructuras en memoria.

Bibliografía

[1] Aditya Bhargava. *Grokking Algorithms.* Manning Publications, 2016.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition.* The MIT Press, 3rd edition, 2009.

[3] Narasimha Karumanchi. *Data Structures and Algorithms Made Easy.* CareerMonk Publications, 2011.

[4] Jon Kleinberg and Éva Tardos. *Algorithm Design.* Pearson, 2005.

[5] Robert Sedgewick and Kevin Wayne. *Algorithms.* Addison-Wesley Professional, 4th edition, 2011.